

## بخش اول

### مقدمه

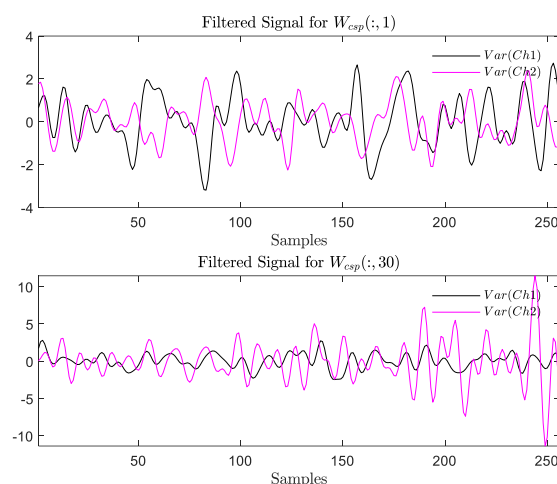
در ابتدا باید میانگین کانال های  $TrainData\_class1$ ,  $TrainData\_class2$  و  $TestData$  را بدست آوریم. برای این کار، روی  $dim$  ۲ و ۳ آن ها  $mean$  می گیریم و سپس نتایج بدست آمده را از خود ماتریس ها کم می کنیم تا میانگین کانال ها صفر شود.

### قسمت الف

برای بدست آوردن فیلتر های مکانی، ابتدا باید ماتریس همبستگی ( $R_x$ ) را برای متغیر های  $TrainData\_class1$  و  $TrainData\_class2$  بدست می آوریم. چون دیتای ما به فرم  $tensor$  می باشد، از رابطه ریاضی به فرم جمع همبستگی لایه های  $tensor$  به شکل  $R_x^{(r)} = \frac{1}{N_r} \sum_{n=1}^{N_r} X_n^r X_n^{rT}$  استفاده می کنیم. برای پیاده سازی این الگوریتم می توانیم از ضرب تانسوری استفاده کنیم اما به علت حجم زیاد داده در آن شرایط (۱.۵ گیگابایت برای محاسبه ضرب) از حلقه  $for$  استفاده می کنیم. سپس ماتریس همبستگی برای این دو متغیر را به ترتیب  $Rx\_1$  و  $Rx\_2$  نام گذاری می کنیم. حال با استفاده از دستور  $eig(Rx\_1, Rx\_2)$  در متلب، ماتریس مقادیر ویژه و ماتریس بردار های ویژه را بدست می آوریم. مشابه پروژه قبلی، چون ماتریس مقادیر ویژه ما به ترتیب صعودی می باشد، آن را با دستور  $sort()$  به نحوی تغییر می دهیم تا ترتیب آن نزولی شود. ماتریس مقادیر ویژه ( $D$ ) به صورت زیر می باشد:

$$D = \begin{bmatrix} 2.2494 & 0 & 0 & \dots & 0 & 0 \\ 0 & 1.4902 & 0 & \dots & 0 & 0 \\ 0 & 0 & 1.3556 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0.3085 & 0 \\ 0 & 0 & 0 & \dots & 0 & 0.3010 \end{bmatrix}$$

حال، چون سیگنال فیلتر شده متناظر با فیلتر اول را می خواهیم، ستون اول از ماتریس فیلتر ( $W_{CSP}$ ) را بر می داریم و ترانهاده آن را در سیگنال آزمایش ۴۹ ام کلاس اول و کلاس دوم ضرب می کنیم. حال دو سیگنال واریانس برای کلاس اول و کلاس دوم داریم. همچنین، چون سوال سیگنال فیلتر شده متناظر با ستون سی ام را نیز خواسته، ستون سی ام از ماتریس فیلتر ( $W_{CSP}$ ) را بر می داریم و ترانهاده آن را در سیگنال آزمایش ۴۹ ام کلاس اول و کلاس دوم ضرب می کنیم. واریانس سیگنال حاصل و همچنین مقایسه کانال اول با کانال دوم این دو فیلتر در تصویر ۱.۱ آمده است.



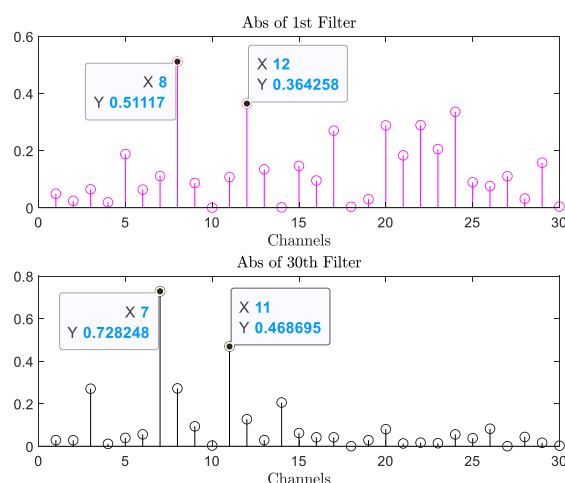
تصویر ۱.۱: واریانس کلاس های اول و دوم برای فیلتر های مکانی ۱ ام و ۳۰ ام (آزمایش ۴۹ ام)

همچنین، این واریانس ها به صورت کمی به صورت زیر می باشند:

$$49th\ experiment \left\{ \begin{array}{l} 1st\ filter \left\{ \begin{array}{l} 1st\ class: 1.406656 \\ 2nd\ class: 0.868848 \end{array} \right. \\ 30th\ filter \left\{ \begin{array}{l} 1st\ class: 1.008859 \\ 2nd\ class: 7.579428 \end{array} \right. \end{array} \right.$$

## قسمت ب

چون تعداد فیلتر های مکانی گسسته می باشند، از دستور  $stem()$  برای رسم قدر مطلق فیلتر های مکانی استفاده می کنیم. نمودار فیلتر های مکانی اول و سی ام در تصویر ۱.۲ آمده است.



تصویر ۱.۲: قدر مطلق فیلتر های مکانی اول و سی ام

با بررسی تصویر ۱.۲، می بینیم که در فیلتر مکانی اول کانال های ۸ ام و ۱۲ ام تاثیر بیشتری در خروجی دارند؛ همچنین در فیلتر مکانی سی ام کانال های ۷ ام و ۱۱ ام تاثیر زیادتری نسبت به بقیه کانال ها دارند.

## قسمت ج

در ابتدا باید فیلترهای مکانی مهم تر را جداسازی کنیم، برای این کار، در متلب دستور زیر را پیاده سازی می کنیم:

$$\text{important\_filters} = [W_{csp}(:, 1:7) \ W_{csp}(:, 24:30)]$$

حال باید برای این ۶۰ آزمایش  $x^{(1)}(t)$  و  $x^{(2)}(t)$  را بدست آوریم. برای این کار، یک حلقه *for* روی ۶۰ آزمایش می زنیم و هر بار روی یک آزمایش، ضرب فیلترهای مکانی روی این آزمایش را اعمال می کنیم و در نهایت واریانس آن ها را می گیریم تا  $x^{(1)}(t)$  و  $x^{(2)}(t)$  بدست آیند. پس از بدست آمدن این دو ماتریس، برای بدست آوردن  $\mu_1$  و  $\mu_2$  از این ماتریس ها میانگین می گیریم. حال برای بدست آوردن  $\Sigma_1$  و  $\Sigma_2$  از رابطه ریاضی زیر استفاده می کنیم:

$$\Sigma_i = \frac{1}{N_i} \sum_{t=1}^{N_i} (x^{(i)}(t) - \mu_i)(x^{(i)}(t) - \mu_i)^T, \quad i = 1, 2$$

از مباحث درس می دانیم که  $W_{LDA}$  به صورت زیر می باشد:

$$W_{lda} = GEVD(\Sigma_1 + \Sigma_2, (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T)$$

دستور *GEVD* متناظر با دستور *eig()* می باشد. ماتریس مقادیر ویژه بدست آمده در این قسمت به صورت زیر می باشد:

$$D = \begin{bmatrix} 3.7805 & 0 & 0 & \dots & 0 & 0 \\ 0 & \approx 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \approx 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & \approx 0 & 0 \\ 0 & 0 & 0 & \dots & 0 & \approx 0 \end{bmatrix}$$

چون درایه های قطری به جز درایه اول همگی تقریباً برابر با صفر می باشند، فقط بردار ویژه اول را بر می داریم (ستون اول) و آن را برابر با  $W_{lda}$  قرار می دهیم. حال طبق فرمول  $\mu^{(1)} = W_{lda}^T \mu_1$  و  $\mu^{(2)} = W_{lda}^T \mu_2$ ،  $\mu^{(1)}$  و  $\mu^{(2)}$  را بدست می آوریم. می دانیم که بهترین آستانه برابر با  $c = \frac{\mu^{(1)} + \mu^{(2)}}{2}$  می باشد. در این سوال، مقدار  $c$  برابر با 2.4061 می باشد. همچنین،  $W_{lda}$  نهایی به صورت زیر می باشد:

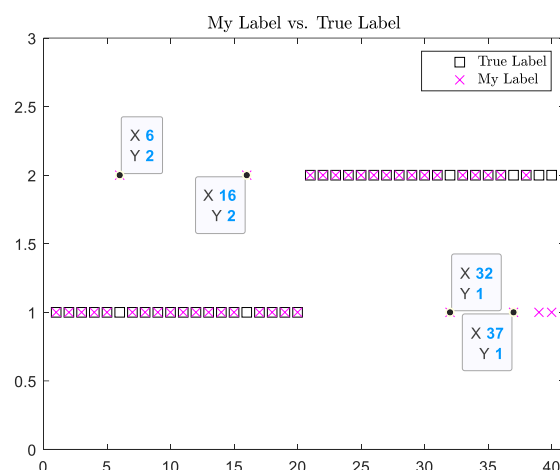
$$W_{lda} = \begin{bmatrix} 0.0052 \\ 0.5123 \\ -7.8671 \times 10^{-4} \\ 0.3894 \\ -0.0075 \\ 0.4989 \\ 0.5197 \\ 0.0295 \\ -0.0306 \\ -0.1383 \\ -0.0182 \\ -0.2058 \\ 0.0462 \\ 0.0316 \end{bmatrix}$$

#### قسمت د

برای این قسمت، باید از متغیر *TestData* استفاده کنیم. این متغیر نیز به صورت *tensor* می باشد بنابراین از حلقه *for* برای محاسبات آن استفاده می کنیم. در حلقه *for*، ابتدا ترانزاده فیلترهای مکانی مهم بدست آمده در قسمت ج را در هر آزمایش ضرب می کنیم و سپس واریانس سیگنال بدست آمده را ذخیره می کنیم. در انتها ۴۰ مقدار داریم. برای این ۴۰ مقدار باید تصمیم گیری کنیم. برای این کار از آستانه ای که بدست آوردیم استفاده می کنیم. چون  $\mu^{(1)}$  از  $\mu^{(2)}$  بیشتر است می توانیم به این نتیجه برسیم که اگر اعداد نیاز به تصمیم گیری از آستانه (*c*) کمتر بودند متعلق به کلاس دوم و اگر این اعداد از آستانه کمتر بودند متعلق به کلاس اول می باشند.

#### قسمت ه

برای این قسمت از دستور *stem()* برای بررسی میزان صحت الگوریتم خود استفاده می کنیم. این نمودار در تصویر ۱.۳ آمده است.



تصویر ۱.۳: مقایسه برچسب‌های صحیح با برچسب‌های زده شده توسط الگوریتم CSP

با بررسی تصویر ۱.۳، مشاهده می‌شود که این الگوریتم در آزمایش‌های 6, 16, 32, 37, 39, 40 به اشتباه برچسب‌گذاری را انجام می‌دهد؛ یعنی ۶ عدد از ۴۰ عدد را به اشتباه برچسب‌گذاری می‌کند. پس خطای الگوریتم برای این مسئله برابر با 15% می‌باشد.

در نتیجه می‌توان گفت که دقت الگوریتم CSP برای این مسئله برابر با 85% می‌باشد.

**بخش اول (انجام با ضرب تانسوری)****توضیحات کلی**

برای اینکه کد بهتر باشد تلاش می‌کنیم تا آنرا به فرمی تغییر دهیم که کمترین حلقه *for* را داشته باشیم. به یاد داریم که در بخش اول حلقه‌های *for* به دلیل اینکه به جای ماتریس، تانسور داشتیم وجود داشتند. همچنین آگاهیم که این ضرب‌ها را برای ضرب صفحه *i* ام تانسور اول در ضرب صفحه *i* ام تانسور دوم را می‌خواستیم. بنابراین تمام ضرب تانسور را نمی‌خواهیم بلکه ضرب صفحه تانسور اول در ضرب صفحه متناظر با آن در تانسور دوم را می‌خواهیم. برای انجام این کار از تابع *pagemtimes()* متلب استفاده می‌کنیم. همچنین، چون ترانهاده تانسور با متد ماتریس بدست نمی‌آید، برای بدست آوردن ترانهاده آن از تابع *pagetranspose()* استفاده می‌کنیم. حال، با مقایسه این کد با کد قبلی مشاهده می‌شود که در این بخش ۳ حلقه *for* (تمام حلقه‌های *for* این بخش) کاهش داشتیم و همچنین در اجرای کد افزایش سرعت خوبی داریم.

(کد‌های با حلقه *for* کمتر در فولدر *less for* و کد‌های با حلقه بیشتر در فولدر *more for* می‌باشند).

## بخش دوم

## مقدمه

در ابتدا باید متغیر زمان را تعریف کنیم، برای این کار طبق گفته سوال باید زمان شروع را برابر با 0 و زمان پایان را برابر با 5 ثانیه در نظر بگیریم و با فرکانس نمونه برداری 250 هرتز تعریف کنیم.

## قسمت الف

در ابتدا باید ماتریس های  $template$  را درست کنیم. برای این کار، در یک حلقه  $for$  این ماتریس ها را تعریف می کنیم (این ماتریس ها به صورت  $X_{ti}$ ,  $i = 1,2,3,4,5$  می باشند). می دانیم این ماتریس ها به فرمت زیر می باشند:

$$X_{ti} = \begin{bmatrix} -\sin(2\pi f_i t) - \\ -\cos(2\pi f_i t) - \\ -\sin(2 \times 2\pi f_i t) - \\ -\cos(2 \times 2\pi f_i t) - \\ \vdots \end{bmatrix}, \quad i = 1,2,3,4,5$$

سپس این ماتریس را با دستور  $cat()$  به نحوی به هم متصل می کنیم تا ماتریس ما به یک  $tensor$  تبدیل می شود. این کار را برای این انجام می دهیم که بتوانیم در یه حلقه  $for$  تو در تو دوتایی تمام عملیات الگوریتم  $CCA$  را انجام دهیم. حال در حلقه بیرونی یک از آزمایش ها و در حلقه داخلی یک صفحه (برش) از تانسور بر می داریم. این صفحه، معادل عبارت  $X^{(fi)}$  در درس می باشد. اکنون ماتریس های همبستگی  $R_{yx}$  و  $R_{xy}$ ،  $R_x$ ،  $R_y$  را بدست می آوریم. می دانیم که این ماتریس ها از معادلات زیر بدست می آیند. همچنین، برای محاسبه  $\Sigma_1$  و  $\Sigma_2$  داریم:

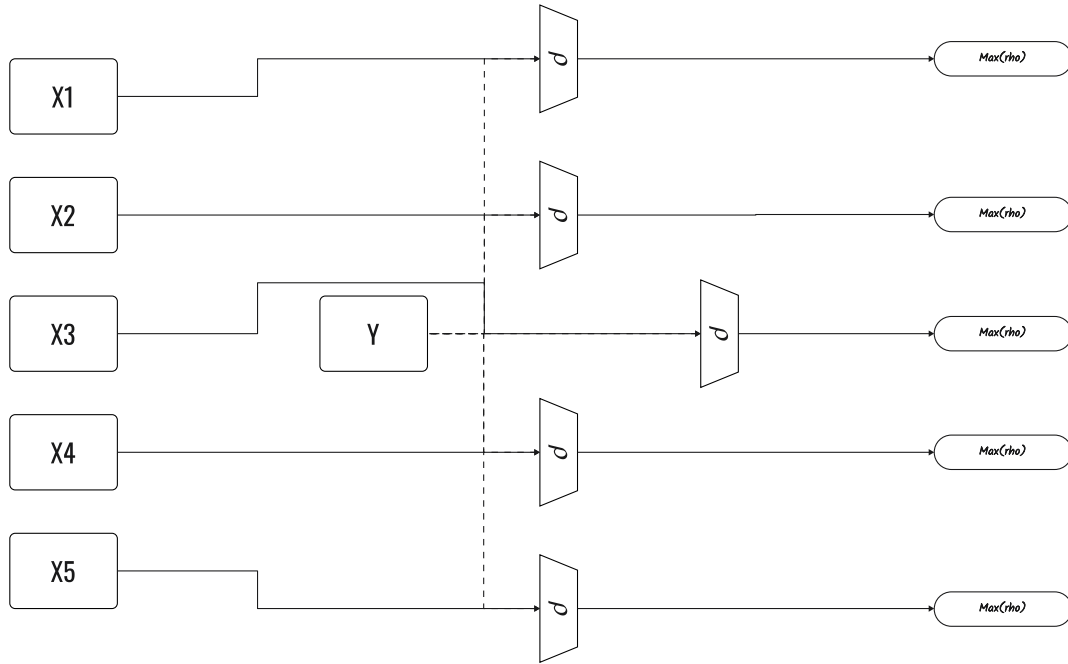
$$\begin{cases} R_x = XX^T \\ R_y = YY^T \\ R_{xy} = XY^T \\ R_{yx} = YX^T \end{cases} \rightarrow \begin{cases} \Sigma_1 = R_x^{-\frac{1}{2}} R_{xy} R_y^{-\frac{1}{2}} \\ \Sigma_2 = R_y^{-\frac{1}{2}} R_{yx} R_x^{-\frac{1}{2}} \end{cases}$$

اکنون که ماتریس های  $\Sigma_1$  و  $\Sigma_2$  را داریم می توانیم از رابطه  $GEVD(\Sigma, I)$  استفاده کنیم؛ چون که اگر روابط زیر را بررسی کنیم می بینیم که در انتها به نسبت رایی رسیده ایم:

$$\begin{aligned} a, b &= \operatorname{argmax}_{a,b} |\rho(a^T X, b^T Y)| \rightarrow \rho = \frac{|a^T X Y^T b|}{\sqrt{a^T X X^T a} \sqrt{b^T Y Y^T Y}} \rightarrow \\ &\xrightarrow{\text{تغییر متغیر}} \begin{cases} c = R_x^{-\frac{1}{2}} a \\ y = R_y^{-\frac{1}{2}} b \end{cases} \rightarrow \rho = \frac{|c^T R_x^{-\frac{1}{2}} R_{xy} R_y^{-\frac{1}{2}} d|}{\sqrt{c^T c} \sqrt{d^T d}} \xrightarrow{e^T = c^T R_x^{-\frac{1}{2}} R_{xy} R_y^{-\frac{1}{2}}} |\rho| < \frac{\sqrt{e^T e}}{\sqrt{c^T c}} \\ &\rightarrow |\rho|^2 = \frac{c^T \Sigma c}{c^T I c} \Rightarrow \text{Rayleigh Quotient} \end{aligned}$$

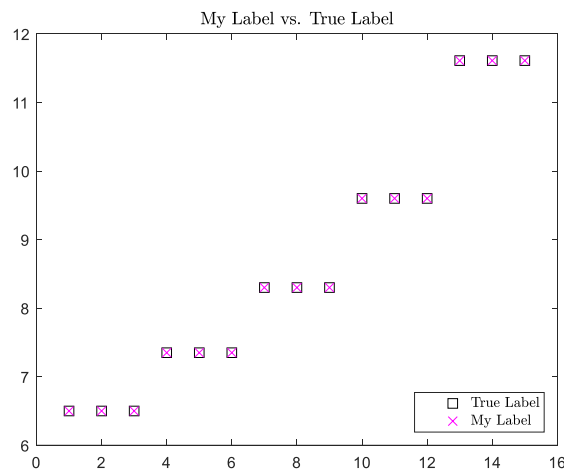
$$\rightarrow \begin{cases} c : GEVD(\Sigma_1, I) \Rightarrow c \xrightarrow{yields} a \left( a = R_x^{-\frac{1}{2}} c \right) \\ d : GEVD(\Sigma_2, I) \Rightarrow d \xrightarrow{yields} b \left( b = R_y^{-\frac{1}{2}} d \right) \end{cases}$$

شماتیک کلی الگوریتم  $CCA$  برای این مسئله در تصویر ۲.۱ آمده است.



تصویر ۲.۱: شماتیک کلی الگوریتم  $CCA$  برای بخش ۲

می‌دانیم که برای هر آزمایش ۵ عدد  $a$  و ۵ عدد  $b$  داریم. حال اگر در فرمول‌های بالا جایگذاری کنیم برای هر آزمایش به ۵ عدد  $\rho$  می‌رسیم، ما به دنبال اندیس ماکزیمم این مقدار هستیم پس با آرگومان دوم دستور  $max$  به این اندیس دست می‌یابیم. سپس، اندیس بدست آمده را در ماتریس داده شده  $freq$  جایگذاری می‌کنیم تا فرکانسی که حدس می‌زنیم درست باشد را بدست بیاریم. در انتها در تصویر ۲.۲ برچسب ۱۵ آزمایش و تفاوت این برچسب‌های زده شده با برچسب واقعی آمده است.



تصویر ۲.۲: مقایسه برچسب‌های صحیح با برچسب‌های زده شده توسط الگوریتم  $CCA$



با بررسی تصویر ۲.۲، می‌بینیم که در هیچ نقطه‌ای برچسب‌های زده شده توسط الگوریتم  $CCA$  متفاوت با برچسب‌های واقعی نیستند.

در نتیجه می‌توان گفت دقت الگوریتم  $CCA$  برای این مسئله برابر با 100% می‌باشد.

## بخش دوم (انجام با ضرب تانسوری)

## توضیح کلی

برای اینکه کد بهتر باشد تلاش می‌کنیم تا آنرا به فرمی تغییر دهیم که کمترین حلقه *for* را داشته باشیم. به یاد داریم که در بخش دوم حلقه‌های *for* به دلیل اینکه به جای ماتریس، تانسور داشتیم وجود داشتند. در این بخش همچنین محاسبات متغیرهای *c* و *d* به روش تانسوری بسیار پیچیده می‌باشند (به دلیل اینکه باید تانسور را به توان برسانیم)؛ بنابراین مجبوریم از حلقه *for* استفاده کنیم (در این بخش تعداد حلقه‌های *for* ما کاهش نمی‌یابند). همچنین آگاهییم که این ضرب هارا برای ضرب صفحه *i* ام تانسور اول در ضرب صفحه *i* ام تانسور دوم را می‌خواستیم. بنابراین تمام ضرب تانسور را نمی‌خواهیم بلکه ضرب صفحه تانسور اول در ضرب صفحه متناظر با آن در تانسور دوم را می‌خواهیم. برای انجام این کار از تابع `pagetimes()` متلب استفاده می‌کنیم. همچنین، چون ترانهاده تانسور با متد ماتریس بدست نمی‌آید، برای بدست آوردن ترانهاده آن از تابع `pagetranspose()` استفاده می‌کنیم. حال، با مقایسه این کد با کد قبلی مشاهده می‌شود که در این بخش تعداد حلقه‌های *for* کاهشی نداشتند؛ اما در اجرای کد، افزایش سرعت قابل توجهی داریم.

(کدهای با حلقه *for* کمتر در فولدر *less for* و کدهای با حلقه بیشتر در فولدر *more for* می‌باشند.)